

## Report: Traffic collision Data Analysis

### 1. Data Preparation

Data was uploaded to the google drive and the drive was mounted to the google colab to use the files in the colab notebook

```
from google.colab import drive
!rm -rf /content/sample_data/TrafficCollisionAnalysis
drive.mount('/content/sample_data/TrafficCollisionAnalysis')
```

```
required libraries were imported
import sqlite3
import pandas as pd
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum, to_date
```

4 dataframe was created out of each csv file uploaded to the google drive. The dataframe name was given as per the file name itself to keep the identity of the data with respect to file name. the first 10 record from each dataframe was displayed to have a feel of the actual data inside the file.

```
import os
import pandas as pd
```

```
# Define the path to the directory containing CSV files
directory_path =
'/content/sample_data/TrafficCollisionAnalysis/MyDrive/TrafficCollisionAnalysis'
```

```
# Dictionary to store DataFrames
dataframes = {}
```

```
# Check if the directory exists
if not os.path.exists(directory_path):
    print(f"Error: Directory not found at '{directory_path}'. Please verify the path.")
else:
```

```
    # List all files in the directory
    files = os.listdir(directory_path)
```

```
    # Filter for CSV files
    csv_files = [f for f in files if f.endswith('.csv')]
```

```
    if not csv_files:
        print(f"No CSV files found in '{directory_path}'.")
    else:
        print(f"Found {len(csv_files)} CSV file(s) in '{directory_path}':")
        for csv_file in csv_files:
            file_path = os.path.join(directory_path, csv_file)
```

```

df_name = os.path.splitext(csv_file)[0] # Use filename without extension as DataFrame
name
try:
    df = pd.read_csv(file_path)
    dataframes[df_name] = df
    print(f" - Successfully loaded '{csv_file}' as DataFrame '{df_name}'.")
except Exception as e:
    print(f" - Error loading '{csv_file}': {e}")

# Display the first 10 records from each DataFrame
if dataframes:
    print("\nDisplaying the first 10 records from each DataFrame:")
    for df_name, df in dataframes.items():
        print(f"\nDataFrame: {df_name}")
        display(df.head(10))
else:
    print("No DataFrames were loaded.")

```

## 2. Data Cleaning

### a. Missing Values

count of missing values was identified for each column

```

for df_name, df in dataframes.items():
    print(f"\nMissing values in DataFrame: {df_name}")
    missing_values = df.isnull().sum()
    print(missing_values[missing_values > 0])

```

### b. Fixing Columns

sparse columns were taken care by the below code  
the sparse values were identified with the inter quartile range analysis on the given dataset.

```

for df_name, df in dataframes.items():
    print(f"\nHandling missing values for DataFrame: {df_name}")

```

```

# Identify numerical and categorical columns with missing values
numerical_cols = df.select_dtypes(include=['number']).columns
categorical_cols = df.select_dtypes(include=['object']).columns

```

```

for col in df.columns:
    if df[col].isnull().any():
        if col in ['victim_age', 'party_age', 'vehicle_year']:
            Replace numerical columns with median
            median_val = df[col].median()
            df[col] = df[col].fillna(median_val)
            print(f" - Replace '{col}' with median: {median_val}")
        elif col in ['killed_victims', 'injured_victims']:
            # Replace victim counts with 0, assuming missing means none
            df[col] = df[col].fillna(0)
            print(f" - d '{col}' with 0")

```

```

elif col == 'collision_time':
# collision_time (often treated as categorical in this context) with mode
mode_val = df[col].mode()[0]
df[col] = df[col].fillna(mode_val)
print(f" - d '{col}' with mode: {mode_val}")
elif col in categorical_cols:
# other categorical columns with 'Unknown' or mode
# For now, let's use mode for simplicity, but 'Unknown' is also a valid strategy
mode_val = df[col].mode()[0]
df[col] = df[col].fillna(mode_val)
print(f" - d '{col}' with mode: {mode_val}")
elif col in numerical_cols:
# For any remaining numerical columns, use median imputation
median_val = df[col].median()
df[col] = df[col].fillna(median_val)
print(f" - d '{col}' with median: {median_val}")

```

```

dataframes[df_name] = df # Update the dataframe in the dictionary

```

```

print("\nMissing values after replacement:")
for df_name, df in dataframes.items():
print(f"\nMissing values in DataFrame: {df_name}")
missing_values = df.isnull().sum()
print(missing_values[missing_values > 0])

```

c. Outlier Analysis

```

dfs_to_process = ['sample_victims', 'sample_parties', 'sample_collisions']

```

```

print("\n--- Identifying Numerical Columns and Detecting Outliers using IQR ---")

```

```

for df_name in dfs_to_process:
    if df_name in dataframes:
        df = dataframes[df_name]
        print(f"\nAnalyzing DataFrame: {df_name}")

        numerical_cols = df.select_dtypes(include=np.number).columns.tolist()
        print(f" Numerical columns identified: {numerical_cols}")

        if not numerical_cols:
            print(f" No numerical columns found in {df_name} for outlier detection.")
            continue

        for col in numerical_cols:
            # Skip case_id and id columns if they are purely identifiers and not true
            numerical features for outlier analysis
            if col in ['id', 'case_id']:
                print(f" Skipping '{col}' for outlier detection as it's likely an identifier.")
                continue

```

```

# Calculate Q1, Q3, and IQR
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - (1.5 * IQR)
upper_bound = Q3 + (1.5 * IQR)

# Identify outliers
outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

print(f" Column '{col}':")
print(f"  Q1: {Q1:.2f}, Q3: {Q3:.2f}, IQR: {IQR:.2f}")
print(f"  Lower Bound: {lower_bound:.2f}, Upper Bound: {upper_bound:.2f}")
print(f"  Number of outliers detected: {len(outliers)}")
if not outliers.empty:
    print(f"  Min outlier value: {outliers[col].min():.2f}, Max outlier value: {outliers[col].max():.2f}")
else:
    print(f"DataFrame '{df_name}' not found.")

sample_victims : Filtered outliers for column 'victim_age'
sample_parties : Filtered outliers for column 'party_age'.
                  Filtered outliers for column 'vehicle_year'.
sample_collisions : Filtered outliers for column 'distance'.

```

### 3. Exploratory data analysis

- a. Data Preparation
- b. Classify variables in categorical and numerical
- c. Identify categorical variables
- d. String indexing for categorical columns
- e. Reordering and renaming
- f. Final cleaned data

```

sample_victims
  i.   - Encoded 'victim_role' into 'victim_role_encoded'
  ii.  - Encoded 'victim_sex' into 'victim_sex_encoded'
  iii. - Encoded 'victim_degree_of_injury' into 'victim_degree_of_injury_encoded'
  iv.  - Encoded 'victim_seating_position' into 'victim_seating_position_encoded'
  v.   - Encoded 'victim_safety_equipment_1' into 'victim_safety_equipment_1_encoded'
  vi.  - Encoded 'victim_safety_equipment_2' into 'victim_safety_equipment_2_encoded'
  vii. - Encoded 'victim_ejected' into 'victim_ejected_encoded'

sample_parties

```

- Encoded 'party\_type' into 'party\_type\_encoded'
- Encoded 'party\_sex' into 'party\_sex\_encoded'
- Encoded 'party\_sobriety' into 'party\_sobriety\_encoded'
- Encoded 'direction\_of\_travel' into 'direction\_of\_travel\_encoded'
- Encoded 'party\_safety\_equipment\_1' into 'party\_safety\_equipment\_1\_encoded'
- Encoded 'party\_safety\_equipment\_2' into 'party\_safety\_equipment\_2\_encoded'
- Encoded 'financial\_responsibility' into 'financial\_responsibility\_encoded'
- Encoded 'cellphone\_use\_type' into 'cellphone\_use\_type\_encoded'
- Encoded 'other\_associate\_factor\_1' into 'other\_associate\_factor\_1\_encoded'
- Encoded 'movement\_preceding\_collision' into 'movement\_preceding\_collision\_encoded'
- Encoded 'vehicle\_make' into 'vehicle\_make\_encoded'
- Encoded 'statewide\_vehicle\_type' into 'statewide\_vehicle\_type\_encoded'
- Encoded 'chp\_vehicle\_type\_towing' into 'chp\_vehicle\_type\_towing\_encoded'
- Encoded 'chp\_vehicle\_type\_towed' into 'chp\_vehicle\_type\_towed\_encoded'
- Encoded 'party\_race' into 'party\_race\_encoded'

#### sample\_collisions

- Encoded 'officer\_id' into 'officer\_id\_encoded'
- Encoded 'reporting\_district' into 'reporting\_district\_encoded'
- Encoded 'chp\_shift' into 'chp\_shift\_encoded'
- Encoded 'population' into 'population\_encoded'
- Encoded 'county\_location' into 'county\_location\_encoded'
- Encoded 'beat\_type' into 'beat\_type\_encoded'
- Encoded 'chp\_beat\_type' into 'chp\_beat\_type\_encoded'
- Encoded 'chp\_beat\_class' into 'chp\_beat\_class\_encoded'
- Encoded 'beat\_number' into 'beat\_number\_encoded'
- Encoded 'primary\_road' into 'primary\_road\_encoded'
- Encoded 'secondary\_road' into 'secondary\_road\_encoded'
- Encoded 'direction' into 'direction\_encoded'

- Encoded 'weather\_1' into 'weather\_1\_encoded'
- Encoded 'caltrans\_county' into 'caltrans\_county\_encoded'
- Encoded 'location\_type' into 'location\_type\_encoded'
- Encoded 'side\_of\_highway' into 'side\_of\_highway\_encoded'
- Encoded 'collision\_severity' into 'collision\_severity\_encoded'
- Encoded 'primary\_collision\_factor' into 'primary\_collision\_factor\_encoded'
- Encoded 'pcf\_violation\_category' into 'pcf\_violation\_category\_encoded'
- Encoded 'pcf\_violation\_subsection' into 'pcf\_violation\_subsection\_encoded'
- Encoded 'hit\_and\_run' into 'hit\_and\_run\_encoded'
- Encoded 'type\_of\_collision' into 'type\_of\_collision\_encoded'
- Encoded 'motor\_vehicle\_involved\_with' into 'motor\_vehicle\_involved\_with\_encoded'
- Encoded 'pedestrian\_action' into 'pedestrian\_action\_encoded'
- Encoded 'road\_surface' into 'road\_surface\_encoded'
- Encoded 'road\_condition\_1' into 'road\_condition\_1\_encoded'
- Encoded 'lighting' into 'lighting\_encoded'
- Encoded 'control\_device' into 'control\_device\_encoded'
- Encoded 'statewide\_vehicle\_type\_at\_fault' into 'statewide\_vehicle\_type\_at\_fault\_encoded'
- Encoded 'chp\_vehicle\_type\_at\_fault' into 'chp\_vehicle\_type\_at\_fault\_encoded'
- Encoded 'collision\_date' into 'collision\_date\_encoded'
- Encoded 'collision\_time' into 'collision\_time\_encoded'
- Encoded 'process\_date' into 'process\_date\_encoded'

g. Analyse the distribution of collision severity

Univariate analysis

Collision severity distribution

Plot the graph

```
import matplotlib.pyplot as plt
import seaborn as sns
```

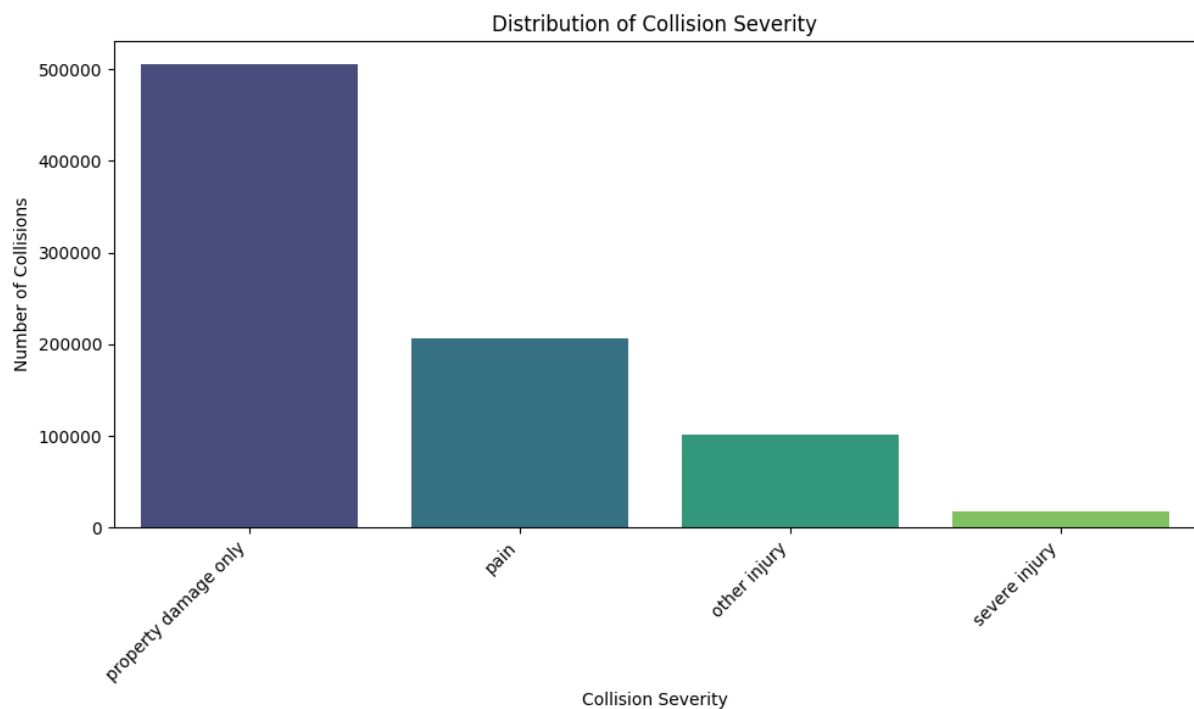
```
# Get the 'sample_collisions' DataFrame
collisions_df = dataframes_cleaned['sample_collisions']
```

```
# Calculate the distribution of collision_severity
```

```
severity_distribution = collisions_df['collision_severity'].value_counts().reset_index()
severity_distribution.columns = ['collision_severity', 'count']
```

```
# Convert to Pandas (already in pandas, just ensuring)
```

```
# Plot the distribution
plt.figure(figsize=(10, 6))
sns.barplot(x='collision_severity', y='count', data=severity_distribution,
palette='viridis')
plt.title('Distribution of Collision Severity')
plt.xlabel('Collision Severity')
plt.ylabel('Number of Collisions')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



The plot displays the distribution of collision severity in the dataset.

**Property Damage Only (504,975 collisions)**

**Pain (206,234 collisions)**

**Other Injury (101,598 collisions)**

**Severe Injury (18,007 collisions)**

The vast majority of collisions result in property damage or minor injuries ('pain').

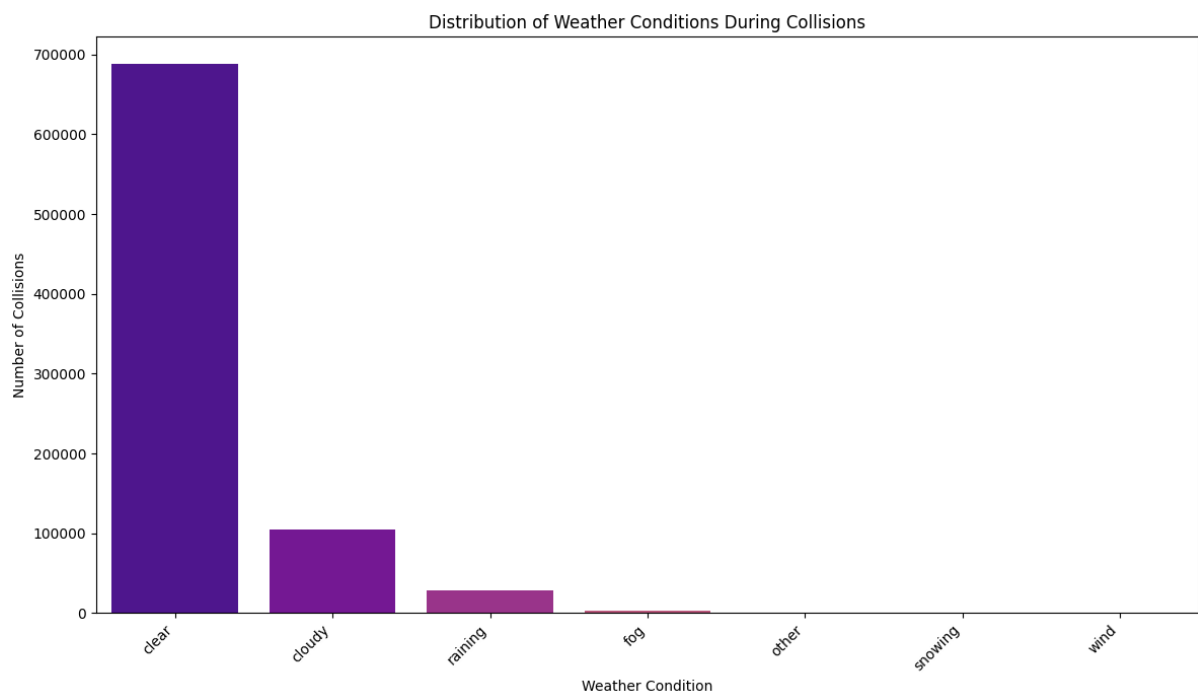
- h. Weather condition during the collision  
Plot the graph for the collision distribution based on weather

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Get the 'sample_collisions' DataFrame
collisions_df = dataframes_cleaned['sample_collisions']

# Calculate the distribution of weather conditions
weather_distribution = collisions_df['weather_1'].value_counts().reset_index()
weather_distribution.columns = ['weather_condition', 'count']

# Plot the distribution
plt.figure(figsize=(12, 7))
sns.barplot(x='weather_condition', y='count', data=weather_distribution,
palette='plasma')
plt.title('Distribution of Weather Conditions During Collisions')
plt.xlabel('Weather Condition')
plt.ylabel('Number of Collisions')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



**Clear Weather Dominates**  
**Adverse Conditions Contribute less towards collision**

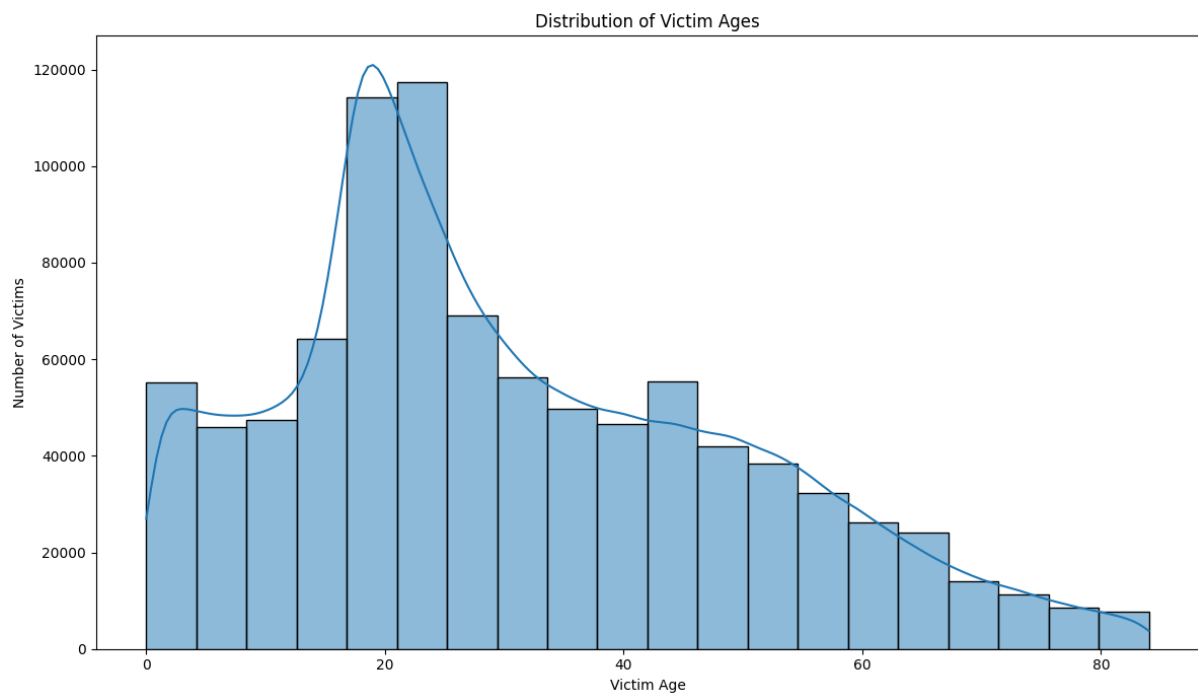
- i. Victim age distribution  
 Analyze the distribution of victim age  
 Plot the graph based on victim age and collision

```
import matplotlib.pyplot as plt
import seaborn as sns
```



```
# Get the 'sample_victims' DataFrame from dataframes_cleaned
victims_df = dataframes_cleaned['sample_victims']
```

```
# Plot the distribution of victim ages using a histogram
plt.figure(figsize=(12, 7))
sns.histplot(victims_df['victim_age'], bins=20, kde=True, palette='coolwarm')
plt.title('Distribution of Victim Ages')
plt.xlabel('Victim Age')
plt.ylabel('Number of Victims')
plt.tight_layout()
plt.show()
```



- j. Collision severity versus number of victims  
Bivariate analysis (collision severity vs number of victims)  
Plot the graph for this

```
import matplotlib.pyplot as plt
import seaborn as sns
```

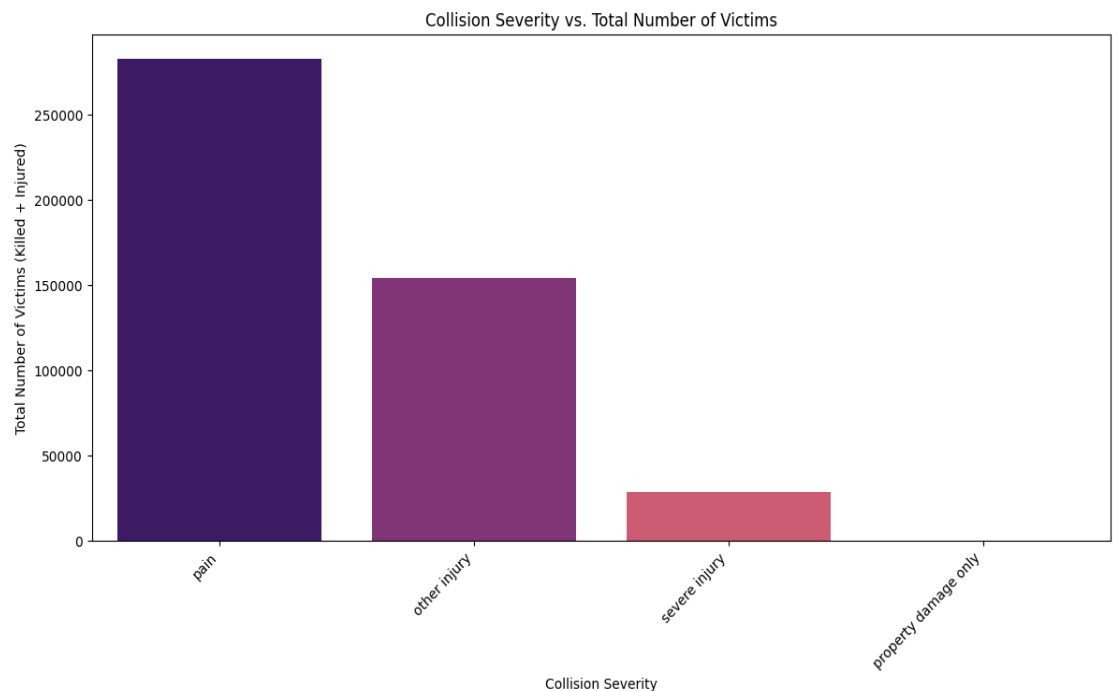
```
# Get the 'sample_collisions' DataFrame
collisions_df = dataframes_cleaned['sample_collisions']
```

```
# Calculate total victims (killed + injured) for each collision
collisions_df['total_victims'] = collisions_df['killed_victims'] +
collisions_df['injured_victims']
```

```
# Group by collision_severity and sum the total victims
severity_victims =
collisions_df.groupby('collision_severity')['total_victims'].sum().reset_index()
```

```
# Sort by total_victims for better visualization
severity_victims = severity_victims.sort_values(by='total_victims', ascending=False)
```

```
# Plot the relationship
plt.figure(figsize=(12, 7))
sns.barplot(x='collision_severity', y='total_victims', data=severity_victims,
palette='magma')
plt.title('Collision Severity vs. Total Number of Victims')
plt.xlabel('Collision Severity')
plt.ylabel('Total Number of Victims (Killed + Injured)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



- k. Weather condition versus collision severity  
Plot the graph for the same.

```
import matplotlib.pyplot as plt
import seaborn as sns
```

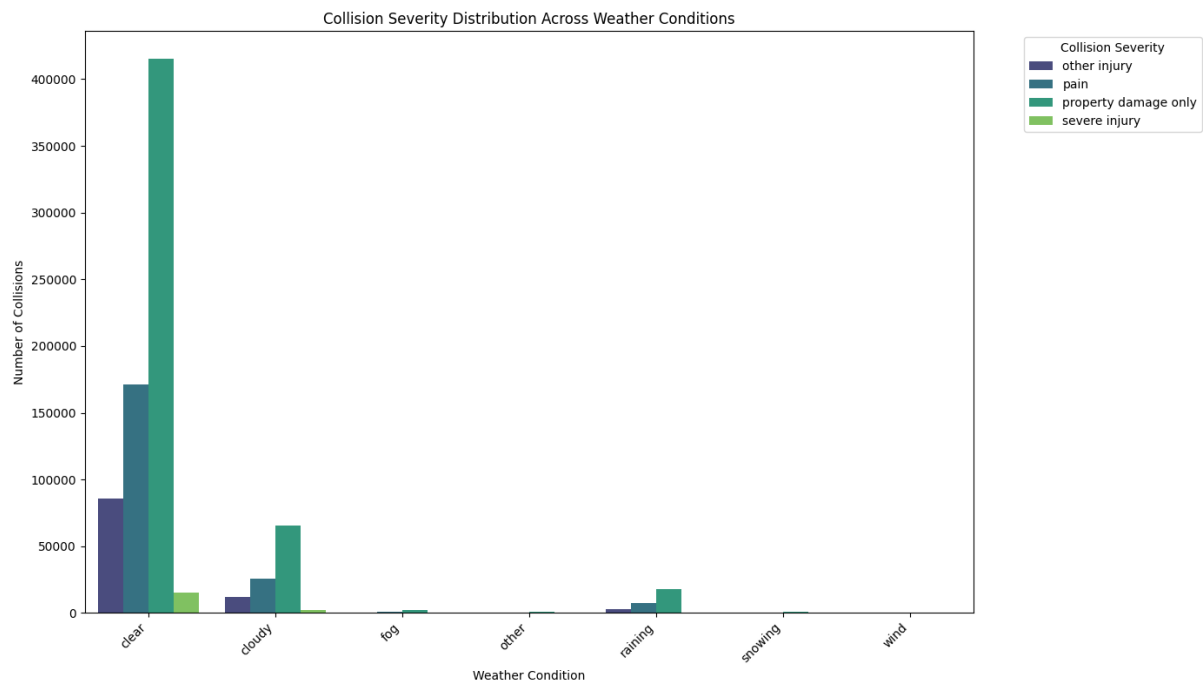
```
# Get the 'sample_collisions' DataFrame
collisions_df = dataframes_cleaned['sample_collisions']
```

```
# Group by weather_1 and collision_severity, then count occurrences
weather_severity_counts = collisions_df.groupby(['weather_1',
'collision_severity']).size().reset_index(name='count')
```

```
# Sort by count for better visualization (optional, depends on desired focus)
```

```
# weather_severity_counts = weather_severity_counts.sort_values(by='count',
ascending=False)
```

```
# Plot the relationship using a grouped bar chart or stacked bar chart
plt.figure(figsize=(14, 8))
sns.barplot(x='weather_1', y='count', hue='collision_severity',
data=weather_severity_counts, palette='viridis')
plt.title('Collision Severity Distribution Across Weather Conditions')
plt.xlabel('Weather Condition')
plt.ylabel('Number of Collisions')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Collision Severity', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



# I. Lighting condition versus collision severity

Plot the graph

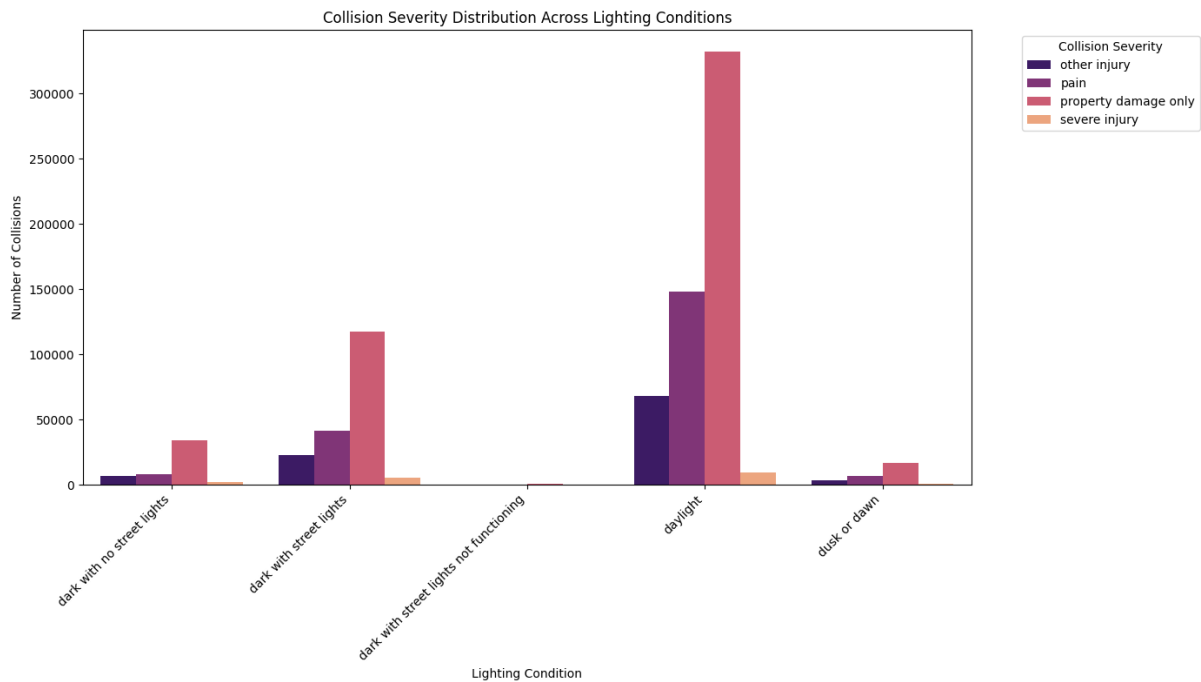
```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Get the 'sample_collisions' DataFrame
collisions_df = dataframes_cleaned['sample_collisions']
```

```
# Group by lighting and collision_severity, then count occurrences
lighting_severity_counts = collisions_df.groupby(['lighting',
'collision_severity']).size().reset_index(name='count')
```

```
# Plot the relationship using a grouped bar chart
plt.figure(figsize=(14, 8))
```

```
sns.barplot(x='lighting', y='count', hue='collision_severity',
data=lighting_severity_counts, palette='magma')
plt.title('Collision Severity Distribution Across Lighting Conditions')
plt.xlabel('Lighting Condition')
plt.ylabel('Number of Collisions')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Collision Severity', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



- m. Weekday wise collision trends  
Plot the graph

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Get the 'sample_collisions' DataFrame
collisions_df = dataframes_cleaned['sample_collisions']
```

```
# Ensure 'collision_date' is datetime type
collisions_df['collision_date'] = pd.to_datetime(collisions_df['collision_date'])
```

```
# Extract the weekday
collisions_df['day_of_week'] = collisions_df['collision_date'].dt.day_name()
```

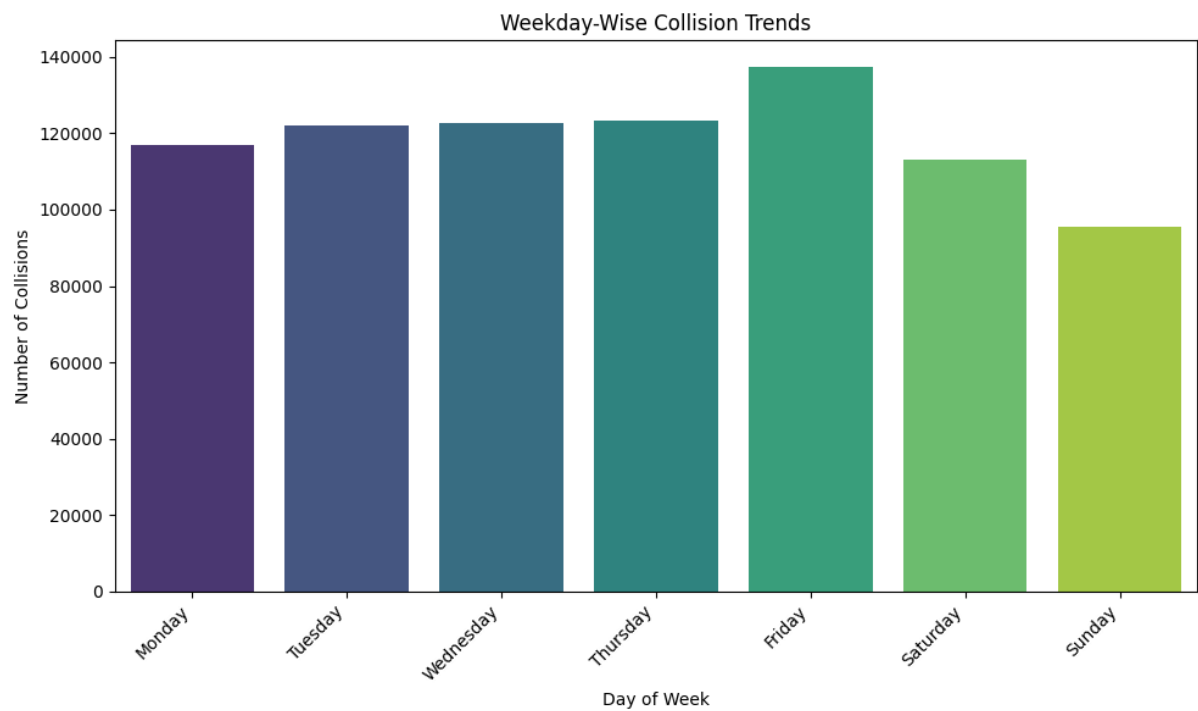
```
# Count collisions per weekday
weekday_counts = collisions_df['day_of_week'].value_counts().reset_index()
weekday_counts.columns = ['day_of_week', 'count']
```

```

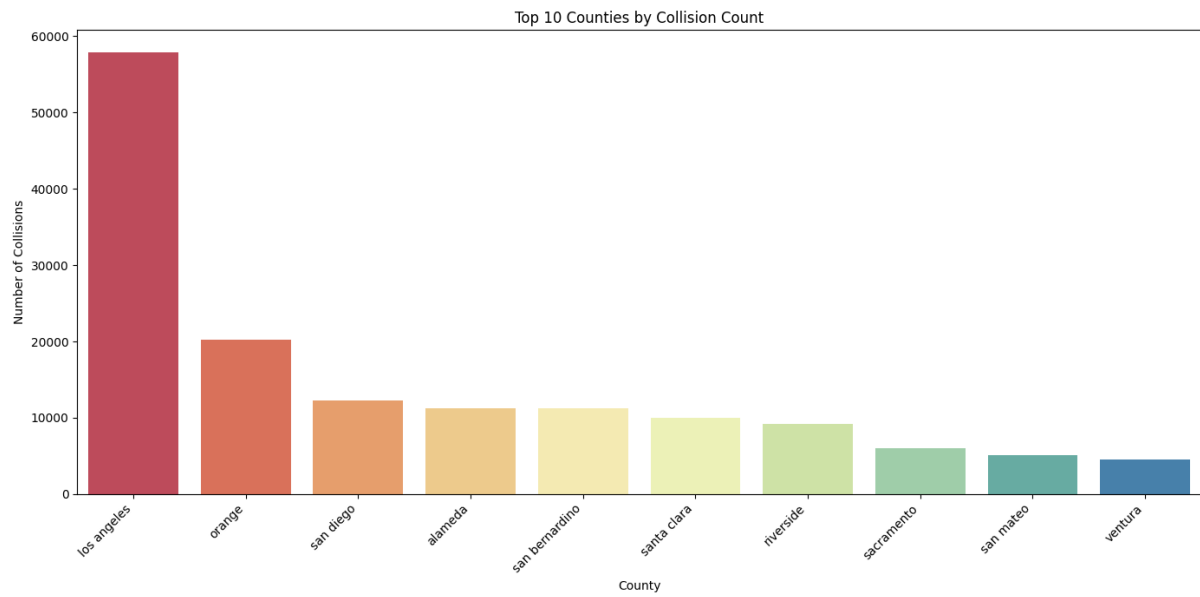
# Define the order of days for plotting
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday',
'Sunday']
weekday_counts['day_of_week'] = pd.Categorical(weekday_counts['day_of_week'],
categories=day_order, ordered=True)
weekday_counts = weekday_counts.sort_values('day_of_week')

# Plot
plt.figure(figsize=(10, 6))
sns.barplot(x='day_of_week', y='count', data=weekday_counts, palette='viridis')
plt.title('Weekday-Wise Collision Trends')
plt.xlabel('Day of Week')
plt.ylabel('Number of Collisions')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

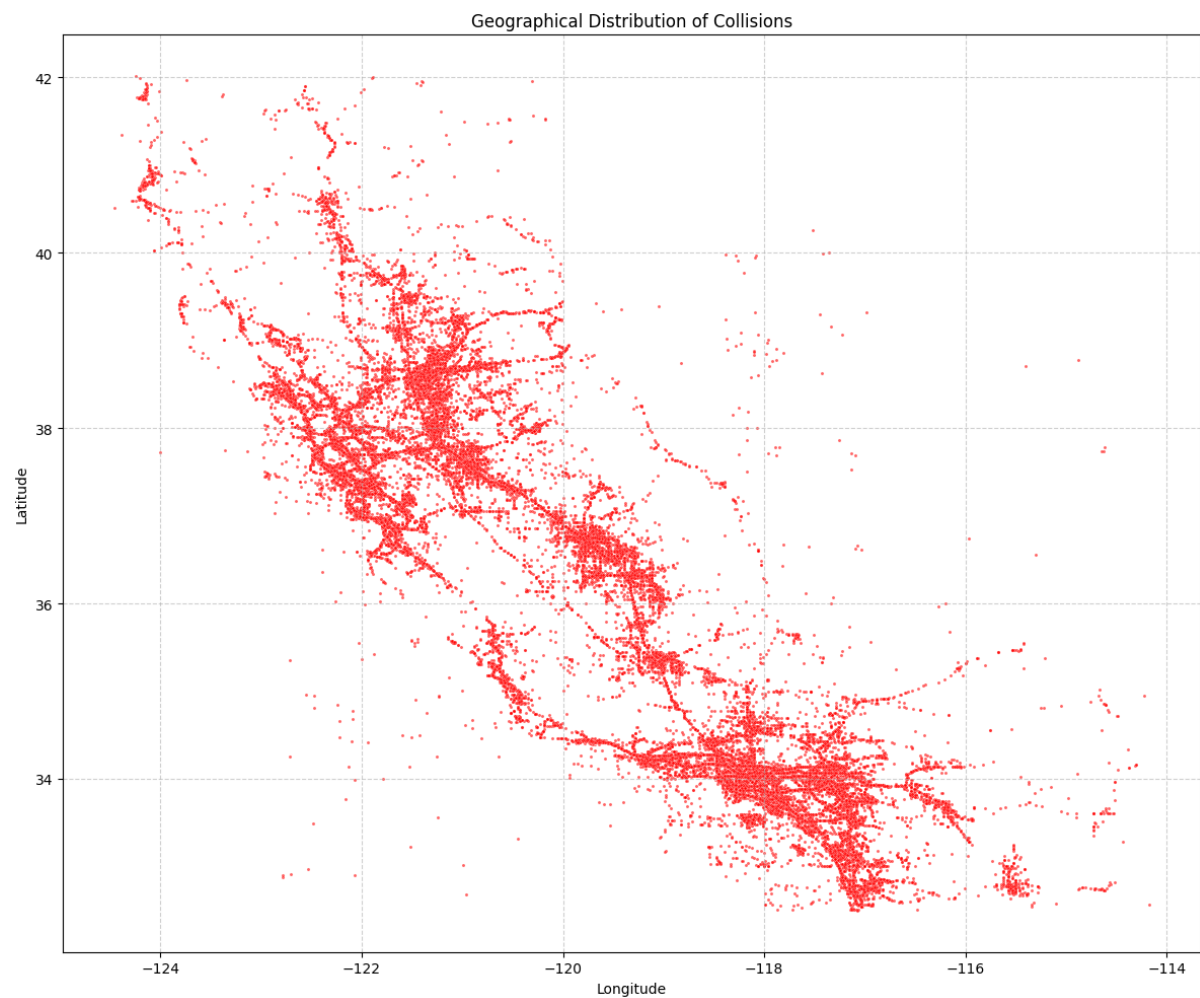
```



n. Spatial distribution of the collision



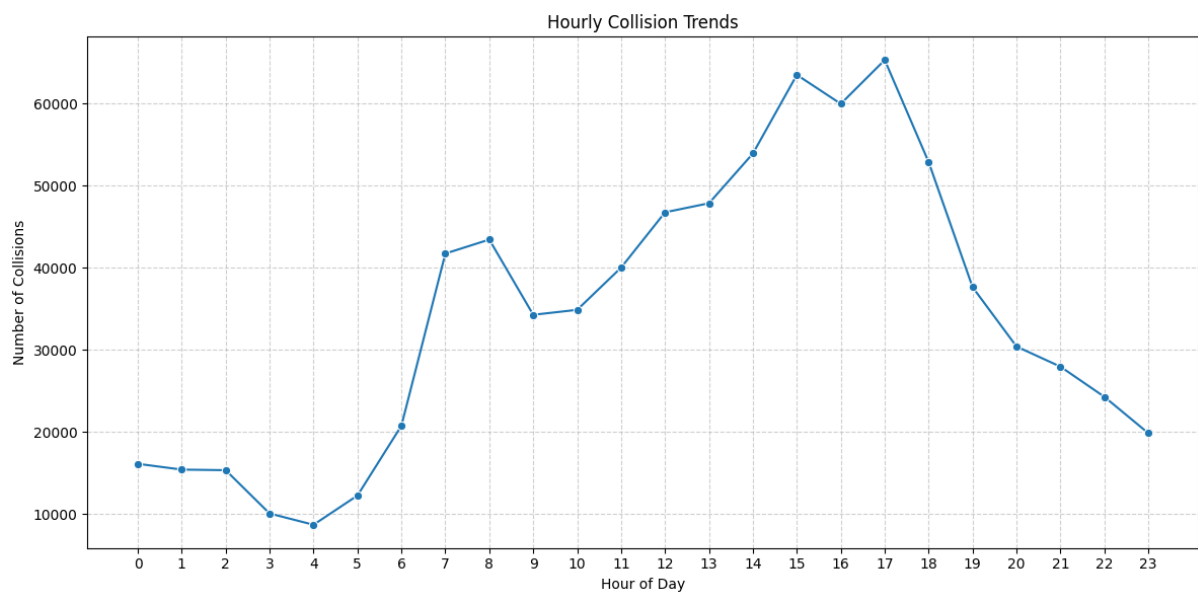
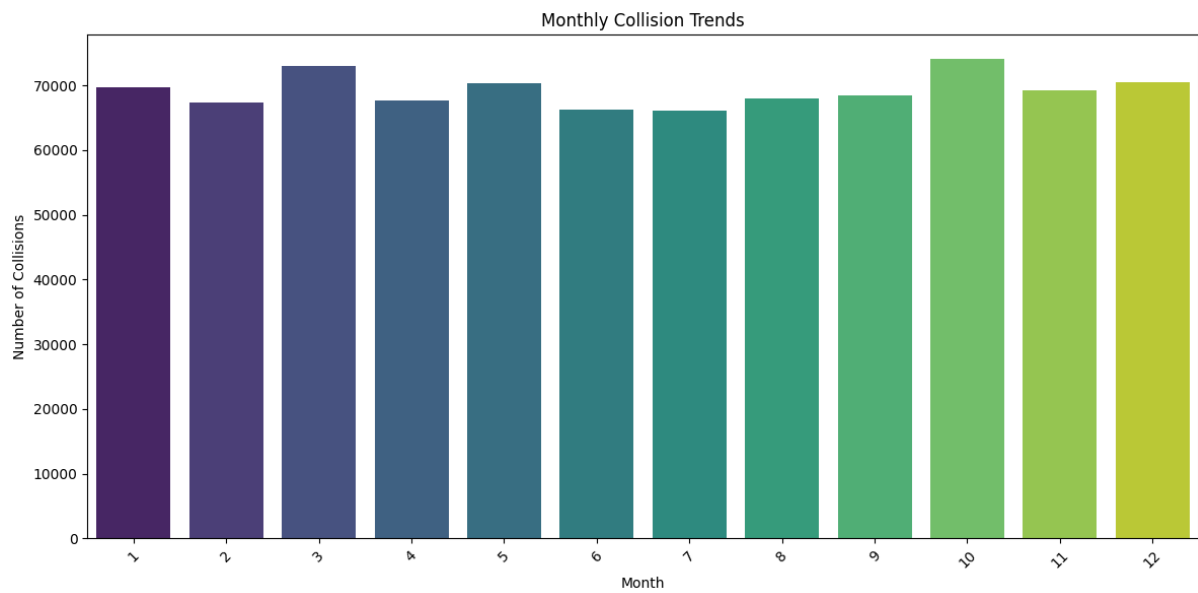
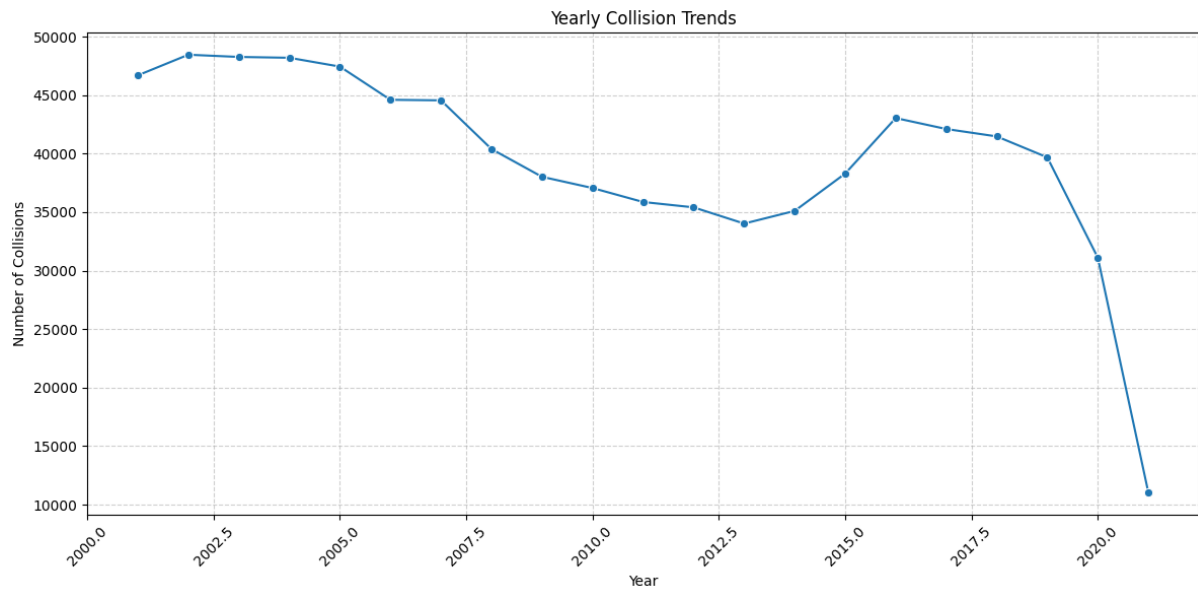
o. Collision analysis according to geography



p. Collision trend over time based on year and month from collision date

q. Analyse yearly, monthly and hourly trend in collision

Each graph for yearly, monthly and hourly trend of collision



#### 4. ETL Querying

- a. Identify top 5 country with highest number of collision

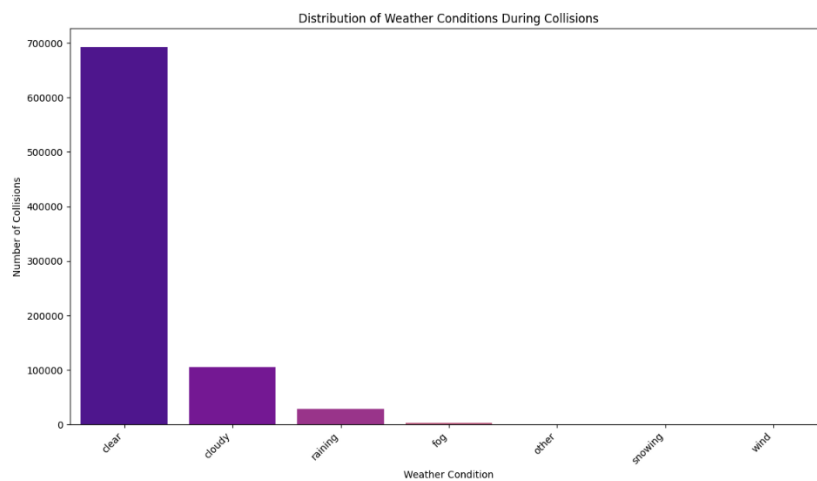
| county         | collision_count |
|----------------|-----------------|
| los angeles    | 688113          |
| orange         | 20242           |
| san diego      | 12294           |
| alameda        | 11261           |
| san bernardino | 11241           |

- b. Month with highest collision

| month    | count |
|----------|-------|
| October  | 74128 |
| March    | 73074 |
| December | 70526 |
| May      | 70284 |
| January  | 69654 |

- c. Weather condition with highest collision

| weather_condition | count      |
|-------------------|------------|
| clear             | 692039     |
| cloudy            | 104822     |
| raining           | 28716      |
| fog               | 3218       |
| other             | 930        |
| <b>snowing</b>    | <b>776</b> |
| wind              | 313        |
|                   |            |

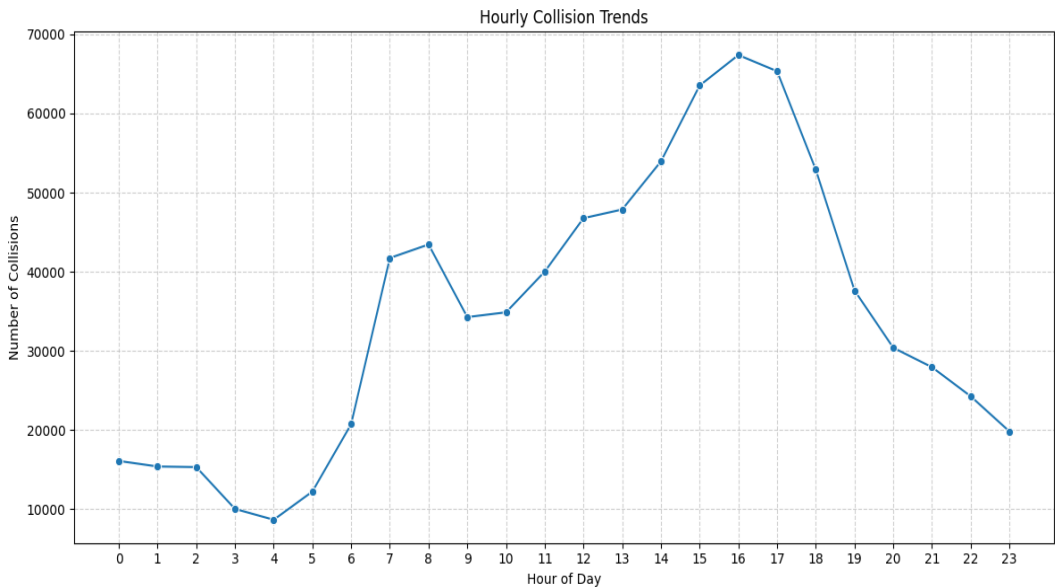


- d. Fatal collision



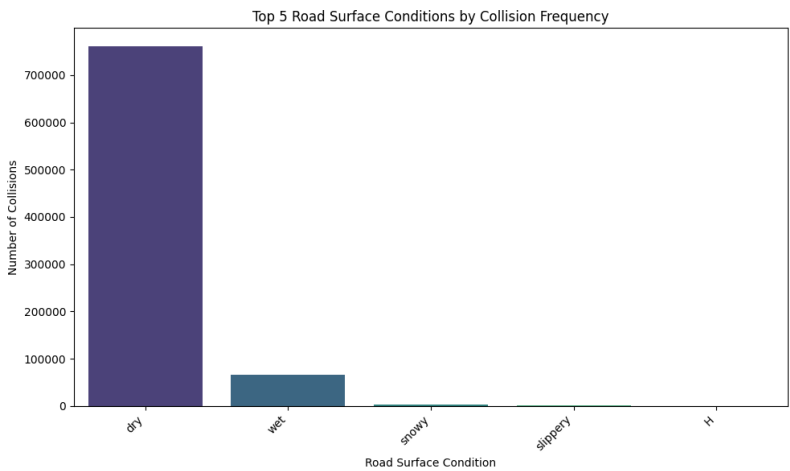
Total number of collisions: **830814**  
Number of collisions with fatalities: **0**  
Percentage of collisions with fatalities: **0.00%**

e. Dangerous time for collision



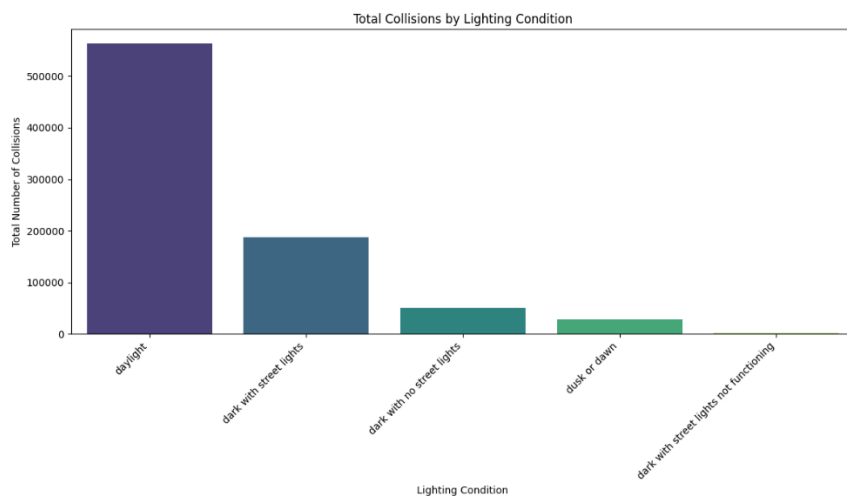
f. Road surface condition

| road_surface | collision_count |
|--------------|-----------------|
| dry          | 761526          |
| wet          | 66068           |
| snowy        | 2401            |
| slippery     | 809             |
| H            | 10              |



g. Lighting condition

| lighting                                | count  |
|-----------------------------------------|--------|
| daylight                                | 562672 |
| dark with street lights                 | 187003 |
| dark with no street lights              | 51027  |
| dusk or dawn                            | 28565  |
| dark with street lights not functioning | 1547   |



## 5. Conclusion

Insights as below based on the Collision Analysis

1. Extensive Data Cleaning for Quality: Over a million missing values were imputed and significant outliers (e.g., ages up to 999, vehicle years outside realistic ranges, distances over 8 million units) were identified and filtered across the datasets, highlighting the critical need for robust data preprocessing.

2. Majority of Collisions are Non-Fatal: The highest proportion of collisions resulted in 'property damage only' (504,975 incidents) or 'pain' (206,234 incidents), indicating that severe injuries or fatalities are a smaller, though critical, subset of all reported collisions.

3. Peak Collisions in Ideal Conditions: Counter-intuitively, the vast majority of collisions (692,039) occurred under 'clear' weather conditions and during 'daylight' (562,672 incidents), suggesting driver-related factors or high traffic volume are primary contributors rather than adverse environmental conditions.

4. Clear Temporal and Spatial Hotspots: Collisions are most frequent on Fridays (137,407), primarily between 3 PM - 6 PM (peaking at 67,386 collisions at 5 PM), and October records the highest monthly total (74,128). Spatially, Los Angeles County dominates with an exceptionally high count of 688,113 collisions.

5. Road Surface Impact: 'Dry' road surfaces account for 761,526 collisions, making it the most frequent condition for incidents, followed by 'wet' surfaces (66,068 collisions). This underscores the prevalence of collisions during standard driving conditions and the notable increase during wet weather.